

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2

з дисципліни «Розробка мобільних
застосувань під Android»

Тема: «ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT»

Виконала:

студентка групи ІА-31

Кизим Єлизавета.

Київ 2026

Тема: Дослідження роботи з компонентом fragment

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набуті практичні навички з використання фрагментів для інтерфейсу користувача.

Завдання:

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

Варіант 12:

12.	Вікно містить групу прапорів (площа і периметр), тобто чек-бокси, групу опцій (різні фігури), тобто радіо-батони, та кнопку «ОК». Вивести інформацію щодо вибору при натисканні на кнопку «ОК» у деяке текстове поле.
-----	---

Хід роботи

Програму розділено на незалежні компоненти, що взаємодіють у межах однієї Діяльності. Передача стану реалізована за допомогою Jetpack ViewModel.

Головна діяльність (MainActivity.kt):

Слугує єдиним контейнером для відображення фрагментів. Відповідає вимозі архітектури "Single-Activity".

Як працює: При старті застосунку метод перевіряє, чи це перший запуск програми (`savedInstanceState == null`). Якщо так, він звертається до диспетчера фрагментів (`supportFragmentManager`), створює транзакцію і завантажує стартовий фрагмент (`InputFragment`) у спеціальний контейнер `FragmentManager`, розміщений у макеті.

Код `MainActivity.kt`:

```
package com.example.lab1

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        if (savedInstanceState == null) {
            supportFragmentManager.beginTransaction()
                .replace(R.id.fragment_container, InputFragment())
                .commit()
        }
    }
}
```

Спільна модель представлення (`SharedViewModel.kt`):

Виконує збереження стану та передачу даних між фрагментами без їхньої прямої взаємодії один з одним, що забезпечує незалежність компонентів (модульність). Як працює: Клас успадковується від `ViewModel` і містить змінну `resultData` типу `String`. Ця змінна зберігає сформований текст результату. Оскільки `ViewModel` прив'язана до життєвого циклу головної діяльності, дані в ній не втрачаються при зміні фрагментів або зміні орієнтації екрана.

Код SharedViewModel.kt:

```
package com.example.lab1

import androidx.lifecycle.ViewModel

class SharedViewModel : ViewModel() {
    var resultData: String = ""
}
```

Фрагмент введення даних (InputFragment .kt):

Виконує відображення форми вибору фігури та параметрів обчислення, а також валідацію даних користувача.

Як працює:

1. Отримує доступ до спільної пам'яті за допомогою `by activityViewModels()`.
2. При натисканні на кнопку "ОК" перевіряє стан елементів форми. Якщо дані не обрано — виводить спливаюче повідомлення `Toast`.
3. Якщо дані заповнені коректно, скрипт формує текстовий рядок результату і записує його у `sharedViewModel.resultData`.
4. Очищення: Знімає виділення з елементів `RadioGroup` та `CheckBox`, готуючи форму до наступного використання.
5. За допомогою `parentFragmentManager` ініціює транзакцію переходу до `ResultFragment`, додаючи поточний екран у стек переходів назад (`addToBackStack`).

Код InputFragment .kt:

```
package com.example.lab1

import android.os.Bundle
import android.view.LayoutInflater
```

```

import android.view.View
import android.view.ViewGroup
import android.widget.*
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels

class InputFragment : Fragment() {

    private val sharedViewModel: SharedViewModel by activityViewModels()

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_input, container, false)

        val radioGroupShapes =
view.findViewById<RadioGroup>(R.id.radioGroupShapes)
        val checkArea = view.findViewById<CheckBox>(R.id.checkArea)
        val checkPerimeter = view.findViewById<CheckBox>(R.id.checkPerimeter)
        val buttonOk = view.findViewById<Button>(R.id.buttonOk)

        buttonOk.setOnClickListener {
            val selectedShapeId = radioGroupShapes.checkedRadioButtonId
            val isAreaChecked = checkArea.isChecked
            val isPerimeterChecked = checkPerimeter.isChecked

            if (selectedShapeId == -1 || (!isAreaChecked &&
!isPerimeterChecked)) {
                Toast.makeText(requireContext(), "Будь ласка, завершіть
введення всіх даних!", Toast.LENGTH_LONG).show()
            } else {
                val shapeRadioButton =
view.findViewById<RadioButton>(selectedShapeId)
                val shapeName = shapeRadioButton.text.toString()

                val selectedParams = mutableListOf<String>()
                if (isAreaChecked) selectedParams.add("Площа")
            }
        }
    }
}

```

```

        if (isPerimeterChecked) selectedParams.add("Периметр")

        val paramsText = selectedParams.joinToString(" та ")

        sharedViewModel.resultData = "Обрана фігура: $shapeName\nОбрані
параметри: $paramsText"

        radioGroupShapes.clearCheck()
        checkArea.isChecked = false
        checkPerimeter.isChecked = false

        parentFragmentManager.beginTransaction()
            .replace(R.id.fragment_container, ResultFragment())
            .addToBackStack(null)
            .commit()
    }
}

return view
}
}

```

Фрагмент результату (ResultFragment.kt):

Відображує отриманні дані на окремому екрані та надає можливості скидання результату (кнопка Cancel).

Як працює: Під час створення візуального інтерфейсу фрагмент звертається до тієї ж SharedViewModel і виводить збережений там текст у компонент TextView. Обробник події для кнопки Cancel робить дві речі: очищає дані в моделі представлення (`sharedViewModel.resultData = ""`) та дає команду системі видалити поточний фрагмент і повернутися на попередній крок за допомогою методу `parentFragmentManager.popBackStack()`.

Код ResultFragment.kt:

```
package com.example.lab1

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels

class ResultFragment : Fragment() {

    private val sharedViewModel: SharedViewModel by activityViewModels()

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_result, container, false)

        val textViewResult = view.findViewById<TextView>(R.id.textViewResult)
        val buttonCancel = view.findViewById<Button>(R.id.buttonCancel)

        textViewResult.text = sharedViewModel.resultData

        buttonCancel.setOnClickListener {
            sharedViewModel.resultData = ""
            parentFragmentManager.popBackStack()
        }

        return view
    }
}
```

Головний макет екрана (activity_main.xml):

Слугує базовим контейнером для всього застосунку, в якому динамічно відображаються фрагменти.

Як працює: Цей файл більше не містить жодних кнопок чи текстових полів.

Замість цього у ньому розміщено компонент `FragmentContainerView` (або `FrameLayout`), який Android використовує як порожнє місце для завантаження та перемикання `InputFragment` та `ResultFragment` під час роботи програми.

Код activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background_soft" />
```

Макет форми введення (fragment_input.xml):

Виконує візуальне представлення першого фрагмента (екрана введення даних), де користувач обирає потрібні параметри.

Як працює: Використовує `ScrollView` для підтримки прокручування на невеликих екранах. Елементи структуровані за допомогою `LinearLayout`. Містить `RadioGroup` з радіо-кнопками для вибору фігури (Квадрат, Коло, Трикутник), два `CheckBox` для вибору параметрів (Площа, Периметр) та кнопку підтвердження «ОК».

Кольори елементів задані через файли ресурсів `@color/`.

Код fragment_input.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```



```
android:layout_height="match_parent"
android:padding="24dp"
android:background="@color/background_soft">
```

```
<LinearLayout
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
```

```
<TextView
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Оберіть фірму:"
    android:textSize="22sp"
    android:textStyle="bold"
    android:textColor="@color/text_header"
    android:layout_marginBottom="12dp"/>
```

```
<RadioGroup
```

```
    android:id="@+id/radioGroupShapes"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="24dp"
    android:padding="8dp">
```

```
<RadioButton
```

```
    android:id="@+id/radioSquare"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:text="Квадрат"
    android:buttonTint="@color/primary_blue" />
```

```
<RadioButton
```

```
    android:id="@+id/radioCircle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="18sp"
```

```

        android:text="Коло"
        android:buttonTint="@color/primary_blue" />

<RadioButton
    android:id="@+id/radioTriangle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:text="Трикутник"
    android:buttonTint="@color/primary_blue" />
</RadioGroup>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Параметри обчислення:"
    android:textSize="22sp"
    android:textStyle="bold"
    android:textColor="@color/text_header"
    android:layout_marginBottom="12dp"/>

<CheckBox
    android:id="@+id/checkArea"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:text="Площа"
    android:buttonTint="@color/check_accent" />

<CheckBox
    android:id="@+id/checkPerimeter"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:text="Периметр"
    android:buttonTint="@color/check_accent"
    android:layout_marginBottom="24dp"/>

```

```

        <Button
            android:id="@+id/buttonOk"
            android:layout_width="match_parent"
            android:layout_height="60dp"
            android:textSize="18sp"
            android:textStyle="bold"
            android:text="OK"
            android:backgroundTint="@color/button_ok_bg"
            android:textColor="@color/white" />

    </LinearLayout>
</ScrollView>

```

Макет екрана результату (fragment_result.xml):

Виконує візуальне представлення другого фрагмента для виводу сформованого результату та надання можливості скидання.

Як працює: Побудований на основі LinearLayout. Містить стилізовану картку з компонентом TextView (@+id/textViewResult), куди програмно передається текст із переліком обраних фігур та параметрів. Також містить кнопку «Cancel» для очищення та повернення.

Код fragment_result.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:background="@color/background_soft">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp"

```

```
android:background="@color/result_card_bg"
android:orientation="vertical">
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Результат:"
    android:textStyle="bold"
    android:textSize="16sp"
    android:textColor="@color/accent_blue" />
```

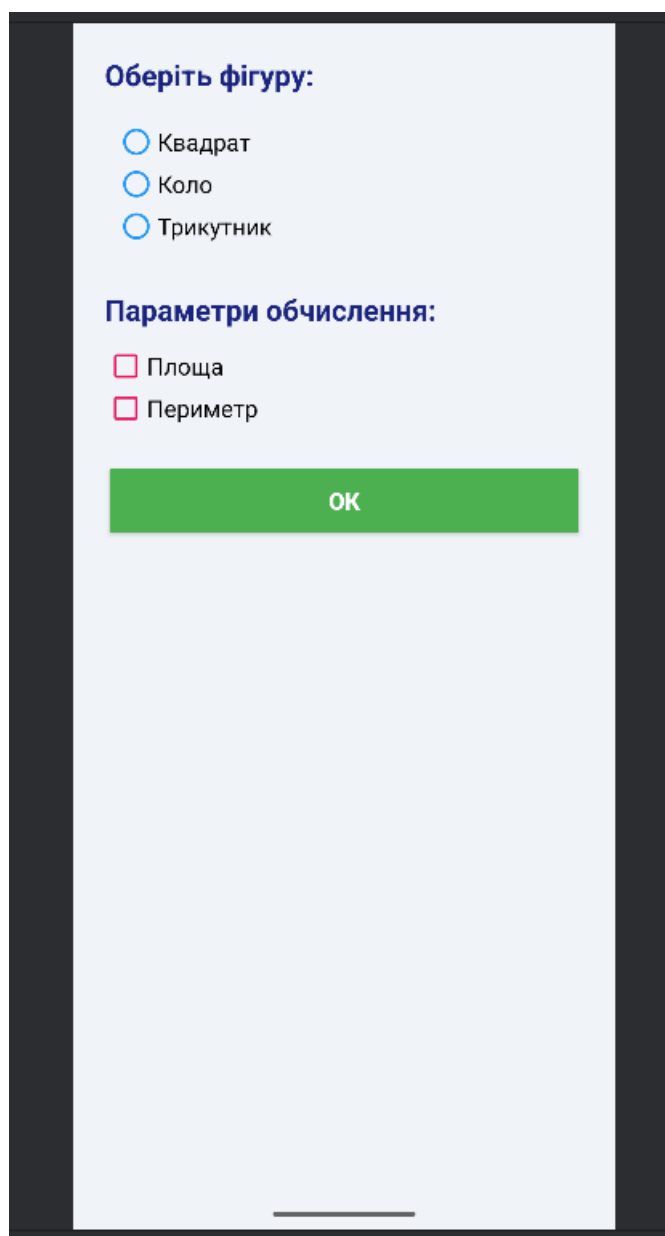
```
<TextView
    android:id="@+id/textViewResult"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:textSize="20sp"
    android:textColor="@color/black"
    android:lineSpacingExtra="4dp" />
```

```
</LinearLayout>
```

```
<Button
    android:id="@+id/buttonCancel"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:layout_marginTop="24dp"
    android:textSize="18sp"
    android:textStyle="bold"
    android:text="Cancel"
    android:backgroundTint="@color/red"
    android:textColor="@color/white" />
```

```
</LinearLayout>
```

Результати роботи програми:



The image shows a mobile application interface with a light blue background and a dark grey border. At the top, the text "Оберіть фігуру:" is displayed in a dark blue font. Below it are three radio button options: "Квадрат", "Коло", and "Трикутник". Further down, the text "Параметри обчислення:" is shown in the same dark blue font. Below this are two checkbox options: "Площа" and "Периметр". At the bottom of the form is a green rectangular button with the text "ОК" in white. The interface is presented within a mobile device frame, with a home indicator bar visible at the very bottom.

Рисунок 1 – Початковий інтерфейс програми: форма введення даних (InputFragment).

Оберіть фігуру:

☐ Квадрат

☐ Коло

☐ Трикутник

Параметри обчислення:

☐ Площа

☐ Периметр

ОК

 Будь ласка, завершіть введення всіх даних!

Рисунок 2 – Робота валідації: відображення Toast-повідомлення при спробі підтвердження з незаповненими даними.

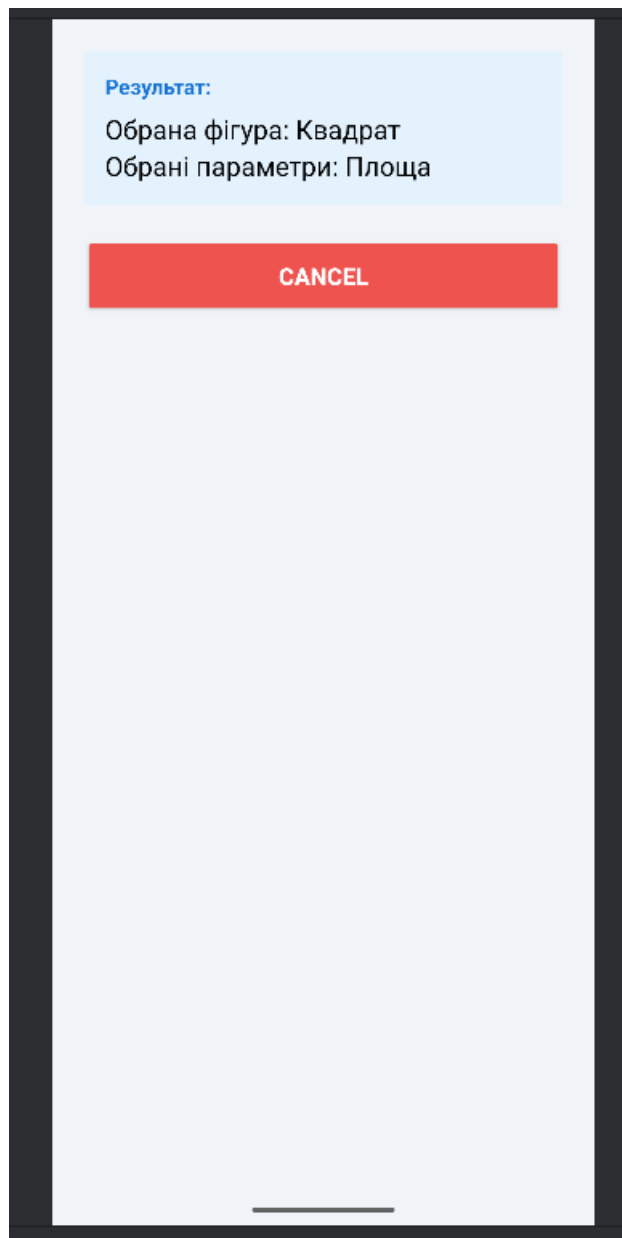


Рисунок 3 – Екран результату (ResultFragment) з виведеними даними щодо обраної фігури та параметрів.

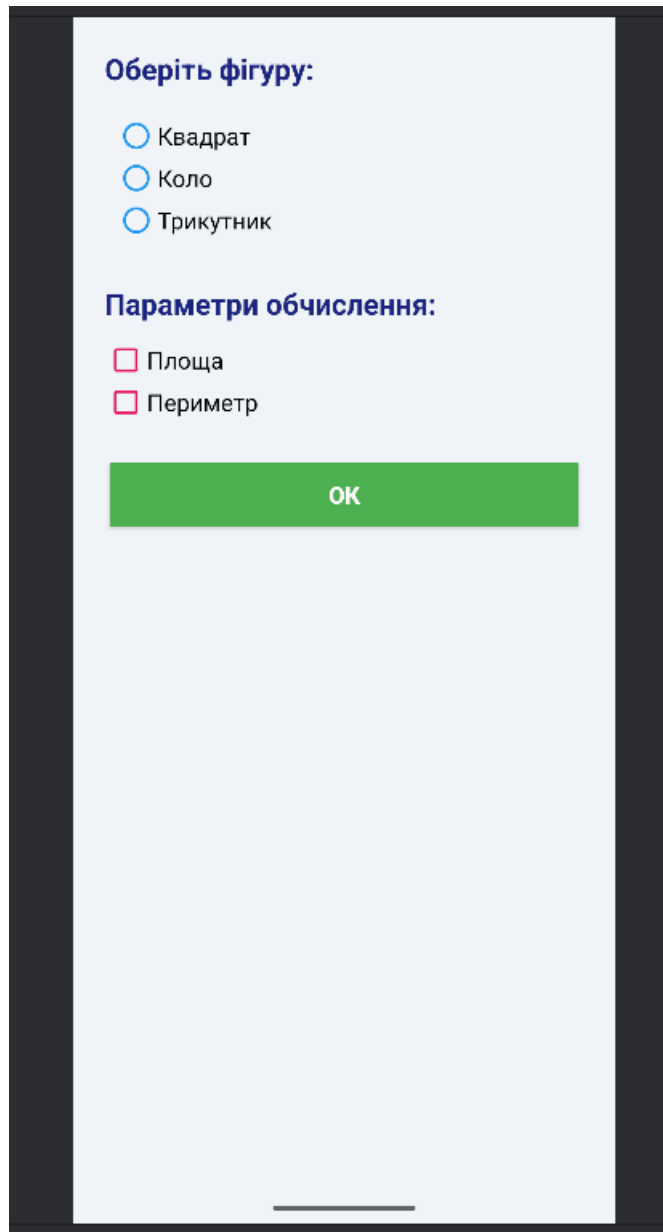


Рисунок 4 – Інтерфейс після натискання кнопки "Cancel": повернення до першого фрагмента з повністю очищеною формою введення.

Висновок: Під час виконання лабораторної роботи було досліджено створення та роботу з компонентом Fragment в ОС Android. Було успішно перенесено логіку застосунку з попередньої роботи на сучасну архітектуру в межах однієї Діяльності. Для взаємодії між фрагментами та надійного збереження стану введених даних між екранами було використано компонент ViewModel. Також було набуто практичних навичок з управління життєвим циклом фрагментів, використання FragmentManager та FragmentTransaction для навігації між екранами (методи replace, addToBackStack, popBackStack). Крім того, згідно з

індивідуальним завданням, було успішно реалізовано функціонал кнопки "Cancel", яка програмно закриває фрагмент результату та гарантує очищення елементів форми на попередньому екрані.

КОНТРОЛЬНІ ПИТАННЯ

1. Призначення та можливості компоненту Фрагмент.

Фрагмент — це модульна частина користувацького інтерфейсу в межах діяльності (Activity). Він призначений для створення гнучких та багаторазових компонентів, які можна додавати, приховувати або замінювати під час роботи програми. Це дозволяє легко адаптувати інтерфейс під різні екрани, наприклад, показувати один фрагмент на смартфоні або два поруч на великому екрані планшета.

2. Життєвий цикл компонента Фрагмент.

Життєвий цикл фрагмента тісно пов'язаний з батьківською діяльністю, але має свої специфічні етапи. Він починається з прив'язки до системи (onAttach) та створення ієрархії відображень (onCreateView). Далі фрагмент стає видимим і готовим до взаємодії з користувачем (onResume). Коли він більше не потрібен або екран перекривається, його візуальна частина знищується (onDestroyView), і він остаточно відв'язується від діяльності (onDetach).

3. Способи створення компонента Фрагмент.

Існує два основні способи створення фрагмента. Статичний спосіб полягає у додаванні фрагмента безпосередньо у файл XML-розмітки діяльності, де його життєвим циклом жорстко керує розмітка. Динамічний (або програмний) спосіб означає створення фрагмента безпосередньо в коді програми та його додавання у спеціальний контейнер під час виконання застосунку за допомогою транзакцій, що дає більше гнучкості.

4. Опишіть способи управління компонентом Фрагмент.

Управління фрагментами здійснюється динамічно за допомогою компонента `FragmentManager` та транзакцій (`FragmentTransaction`). Ці інструменти дозволяють додавати нові фрагменти поверх інших, замінювати поточний фрагмент на новий, видаляти їх, а також приховувати або показувати існуючі фрагменти без їх повного знищення та перестворення.

5. Опишіть способи взаємодії між Фрагментами.

Для збереження модульності фрагменти ніколи не повинні спілкуватися один з одним безпосередньо. Найкращим та рекомендованим способом взаємодії є використання спільної `ViewModel`, яка прив'язана до їхньої спільної `Activity`. Також для передачі результату між фрагментами застосовують механізм `Fragment Result API`, або ж налаштовують комунікацію через саму `Activity` за допомогою реалізації інтерфейсів.

6. Наведіть поняття системи, малої системи та мобільної платформи.

Система загалом — це сукупність взаємопов'язаних елементів, що утворюють цілісність і спільно функціонують для досягнення заданої мети. Мала система — це обчислювальна система зі суттєво обмеженими апаратними ресурсами, такими як об'єм пам'яті, потужність процесора чи заряд батареї. Мобільна платформа являє собою комплексне апаратно-програмне середовище для таких пристроїв, що включає операційну систему, набір бібліотек та інструменти програмування.

7. Опишіть типи мобільних застосунків.

Мобільні застосунки поділяються на кілька типів. Нативні застосунки пишуться під конкретну операційну систему і забезпечують найвищу швидкість та повний доступ до апаратних функцій. Кросплатформові створюються єдиною мовою для різних систем, що економить час розробки. Гібридні функціонують як веб-сторінки всередині прихованого браузера додатка, а прогресивні веб-застосунки (PWA) є оптимізованими сайтами, що запускаються через браузер, але імітують поведінку повноцінних програм.

8. Наведіть класифікацію та загальну характеристику середовищ розробки мобільних застосунків.

Середовища розробки класифікують на спеціалізовані інтегровані середовища (IDE), універсальні редактори та візуальні платформи. Спеціалізовані IDE, такі як Android Studio, надають повний цикл інструментів "з коробки" для нативної розробки. Універсальні редактори коду частіше використовують для кросплатформових проєктів завдяки плагінам. Також існують Low-code та No-code платформи, які дозволяють збирати інтерфейс та логіку візуально з мінімальним написанням коду.

9. Наведіть класифікацію та загальну характеристику мобільних платформ.

Мобільні платформи глобально поділяються на відкриті та закриті. Відкриті платформи, найяскравішим представником яких є Android, мають відкритий вихідний код ядра, що дозволяє різним виробникам техніки вільно адаптувати систему під свої пристрої. Закриті (або пропрієтарні) платформи, такі як iOS, жорстко контролюються однією компанією, яка розробляє і операційну систему, і апаратне забезпечення, що забезпечує високий рівень безпеки та оптимізації.